

- 1.) Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:

1.1 Data type of all columns in the "customers" table.

```
select
column_name, data_type
from scaler-dms1-sql-483907.case_study.INFORMATION_SCHEMA.COLUMNS
where table_name = 'customers';
```

Job information		Results	Visualization	JSON	Execution details	Execution graph
Row	column_name ▼	data_type ▼				
1	customer_id	STRING				
2	customer_unique_id	STRING				
3	customer_zip_code_prefix	INT64				
4	customer_city	STRING				
5	customer_state	STRING				

1.2 Get the time range between which the orders were placed.

```
select
min(order_purchase_timestamp) as first_order_date,
max(order_purchase_timestamp) as last_order_date
from `case_study.orders`;
```

Job information		Results	Visualization	JSON	Execution details
Row	first_order_date ▼	last_order_date ▼			
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC			

1.3 Count the Cities & States of customers who ordered during the given period.

```
select
count(distinct cst.customer_city) as no_of_city,
count(distinct cst.customer_state) as no_of_sate,
from `case_study.customers` as cst
left join `case_study.orders` as ord
```

```
on cst.customer_id = ord.customer_id;
```

Job information				Results	Visualization	JSON	Execution details	Execu
Row	no_of_city	no_of_sate						
1	4119	27						

II. In-depth Exploration:

A. Is there a growing trend in the no. of orders placed over the past years?

```
select
format_date('%Y-%m', order_purchase_timestamp) as
year_month,
count(customer_id) as total_purchase
from `case_study.orders`
group by year_month
order by year_month
;
```



Row	year_month	total_purchase
1	2016-09	4
2	2016-10	324
3	2016-12	1
4	2017-01	800
5	2017-02	1780
6	2017-03	2682
7	2017-04	2404
8	2017-05	3700
9	2017-06	3245
10	2017-07	4026
11	2017-08	4331
12	2017-09	4285
13	2017-10	4631

B. Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

```
select

format_date('%m', order_purchase_timestamp) as mnth,

count(*) as monthly_sales

from `case_study.orders`

group by mnth

ORDER BY mnth desc;
```



Row	mnth	monthly_sales
1	08	10843
2	05	10573
3	07	10318
4	03	9893
5	06	9412
6	04	9343
7	02	8508
8	01	8069
9	11	7544
10	12	5674
11	10	4959

c. During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

- 0-6 hrs : Dawn
- 7-12 hrs : Mornings

- 13-18 hrs : Afternoon
- 19-23 hrs : Night

```
with cte1 as(  
select order_purchase_timestamp, customer_id,  
case when extract(hour from order_purchase_timestamp) between 0 and 6  
then 'Dawn'  
      when extract(hour from order_purchase_timestamp) between 7 and  
12 then 'Morning'  
      when extract(hour from order_purchase_timestamp) between 13 and  
18 then 'Afternoon'  
      when extract(hour from order_purchase_timestamp) between 19 and  
23 then 'Night'  
end as shift  
from `case_study.orders`  
)  
  
select  
shift,  
count(customer_id) as total_orders  
from cte1  
group by shift  
ORDER BY total_orders DESC;
```



Row	shift ▼	total_orders ▼	
1	Afternoon	38135	
2	Night	28331	
3	Morning	27733	
4	Dawn	5242	

III. Evolution of E-commerce orders in the Brazil region:

A. Get the month on month no. of orders placed in each state.

```
select
cst.customer_state,format_date('%m', order_purchase_timestamp) as mnth,
count(ord.order_id) as total_order
from `case_study.customers` as cst
inner join `case_study.orders` as ord
on cst.customer_id = ord.customer_id
group by mnth, cst.customer_state
order by total_order desc
```

Job information		Results	Visualization	JSON	Execution details
Row	customer_state ▼	mnth ▼	total_order ▼		
1	SP	08	4982		
2	SP	05	4632		
3	SP	07	4381		
4	SP	06	4104		
5	SP	03	4047		
6	SP	04	3967		
7	SP	02	3357		
8	SP	01	3351		
9	SP	11	3012		
10	SP	12	2357		
11	SP	10	1908		
12	SP	09	1648		
13	RJ	05	1321		
14	RJ	08	1307		

B. How are the customers distributed across all the states?

```
Select
customer_state, count(customer_id) as total_customer
from `case_study.customers`
group by customer_state
order by total_customer desc
```

Row	customer_state ▼	total_customer ▼
1	SP	41746
2	RJ	12852
3	MG	11635
4	RS	5466
5	PR	5045
6	SC	3637
7	BA	3380
8	DF	2140
9	ES	2033
10	GO	2020

IV. Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

- A. Get the % increase in the cost of orders from year 2017 to 2018 (*include months between Jan to Aug only*).

```

WITH YearlyTotals AS (
    SELECT
        format_date('%Y', o.order_purchase_timestamp) AS
        order_year,
        SUM(p.payment_value) AS total_payment
    FROM `case_study.orders` o
    JOIN `case_study.payments` p ON o.order_id = p.order_id
    WHERE
        format_date('%m', o.order_purchase_timestamp) BETWEEN
        '01' AND '08'
        AND format_date('%Y', o.order_purchase_timestamp) IN
        ('2017', '2018')
    GROUP BY order_year
)
SELECT
    y17.total_payment AS cost_2017,

```



```

y18.total_payment AS cost_2018,
((y18.total_payment - y17.total_payment) /
y17.total_payment) * 100 AS percentage_increase
FROM YearlyTotals y17
JOIN YearlyTotals y18 ON y17.order_year = '2017' AND
y18.order_year = '2018';

```

Row	cost_2017	cost_2018	percentage_incre...
1	3669022.119999...	8694733.840000...	136.9768716466...

B. Calculate the Total & Average value of order price for each state.

```

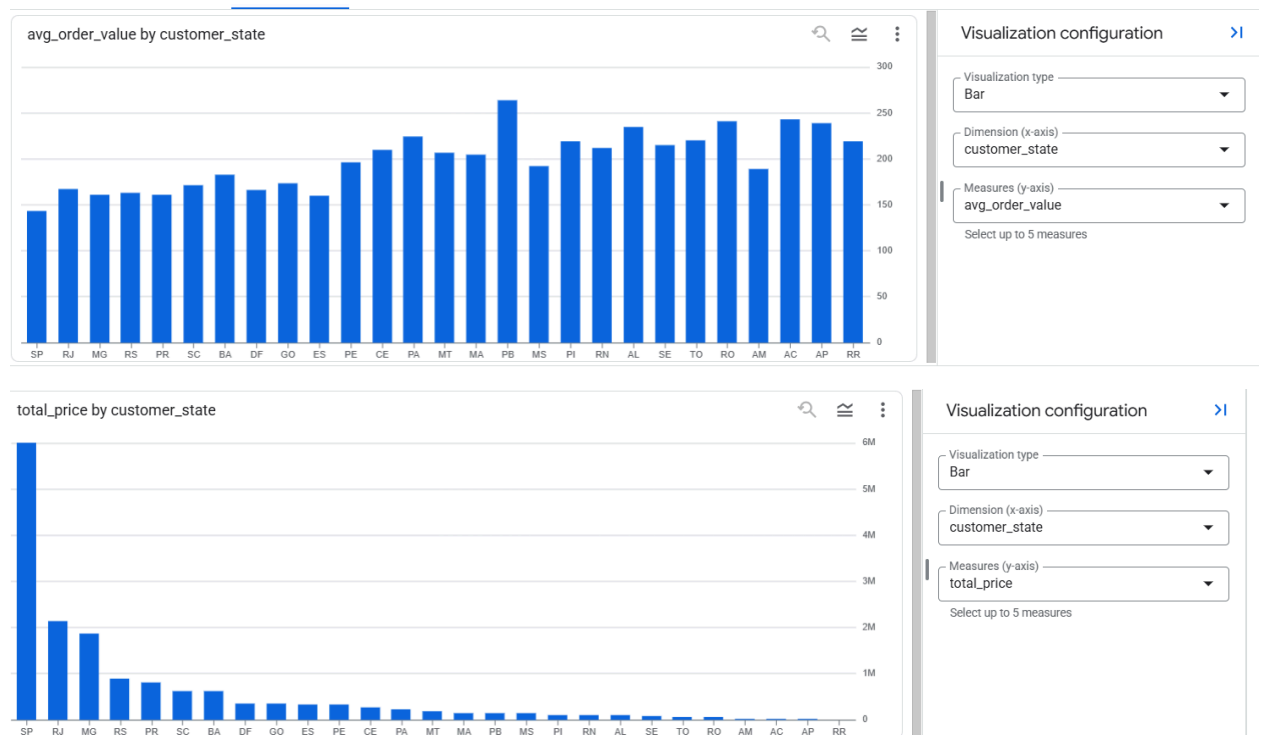
with cte as(
select
c.customer_state,
sum(p.payment_value) as total_price,
count(distinct o.order_id) as order_count
from `case_study.orders` as o
inner join `case_study.payments` as p
on p.order_id = o.order_id

inner join `case_study.customers` as c
on c.customer_id = o.customer_id
group by c.customer_state
)

select
customer_state, round(cte.total_price, 2) as total_price,
round(cte.total_price/cte.order_count, 2) as avg_order_value
from cte
order by total_price desc
;

```

Row	customer_state	total_price	avg_order_value
1	SP	5998226.96	143.69
2	RJ	2144379.69	166.85
3	MG	1872257.26	160.92
4	RS	890898.54	162.99
5	PR	811156.38	160.78
6	SC	623086.43	171.32
7	BA	616645.82	182.44
8	DF	355141.08	165.95
9	GO	350092.31	173.31
10	ES	325967.55	160.34
11	PE	324850.44	196.64



c. Calculate the Total & Average value of order freight for each state.

```

select
c.customer_state,
round(sum(p.freight_value),2) as total_freight_value,
round(sum(p.freight_value)/count(distinct o.order_id),2) as
avg_freight_value
from `case_study.orders` as o
inner join `case_study.order_items` as p
on o.order_id = p.order_id

inner join `case_study.customers` as c
on c.customer_id = o.customer_id
group by c.customer_state
order by total_freight_value desc

```

Row	customer_state	total_freight_value	avg_freight_value
1	SP	718723.07	17.37
2	RJ	305589.31	23.95
3	MG	270853.46	23.46
4	RS	135522.74	24.95
5	PR	117851.68	23.58
6	BA	100156.68	29.83
7	SC	89660.26	24.82
8	PE	59449.66	36.07
9	GO	53114.98	26.46
10	DF	50625.5	23.82
11	ES	49764.6	24.58
12	CE	48351.59	36.44

v. Analysis based on sales, freight and delivery time.

- A. Find the no. of days taken to deliver each order from the order's purchase date as delivery time.

Also, calculate the difference (in days) between the estimated & actual delivery date of an order.

Do this in a single query.

```
SELECT
    order_id,
    DATE_DIFF(order_delivered_customer_date,
order_purchase_timestamp, day) AS time_to_deliver,
    DATE_DIFF(order_delivered_customer_date,
order_estimated_delivery_date, day) AS diff_estimated_delivery
FROM `case_study.orders`
WHERE order_status = 'delivered'
    AND order_delivered_customer_date IS NOT NULL
LIMIT 10;
```

Row	order_id	time_to_deliver	diff_estimated_d...
1	bfb0f9bdef84302105ad712db...	54	36
2	3b697a20d9e427646d9256791...	23	0
3	be5bc2f0da14d8071e2d45451...	24	-10
4	cd3b8574c82b42fc8129f6d502...	10	-39
5	31b0dd6152d2e471443debf03...	8	-40
6	d207cc272675637bfed0062edf...	27	-22
7	98974b076b01553d49ee64679...	43	-6
8	c4b41c36dd589e901f6879f25a...	36	-14
9	a8c3f65a43b2e956cbfca10db4...	15	-34
10	79ffdd52a918bbe867895a4b18...	22	-28

B. Find out the top 5 states with the highest & lowest average freight value.

```
(SELECT
    c.customer_state,
    AVG(oi.freight_value) AS avg_freight,
    'Highest' AS type
FROM `case_study.order_items` oi
```

```

JOIN `case_study.orders` o ON oi.order_id = o.order_id
JOIN `case_study.customers` c ON o.customer_id =
c.customer_id
GROUP BY c.customer_state
ORDER BY avg_freight DESC
LIMIT 5)

UNION ALL

(SELECT
    c.customer_state,
    AVG(oi.freight_value) AS avg_freight,
    'Lowest' AS type
FROM `case_study.order_items` oi
JOIN `case_study.orders` o ON oi.order_id = o.order_id
JOIN `case_study.customers` c ON o.customer_id =
c.customer_id
GROUP BY c.customer_state
ORDER BY avg_freight ASC
LIMIT 5);

```

Row	customer_state	avg_freight	type
1	RR	42.98442307692...	Highest
2	PB	42.72380398671...	Highest
3	RO	41.06971223021...	Highest
4	AC	40.07336956521...	Highest
5	PI	39.14797047970...	Highest
6	SP	15.14727539041...	Lowest
7	PR	20.53165156794...	Lowest
8	MG	20.63016680630...	Lowest
9	RJ	20.96092393168...	Lowest
10	DF	21.04135494596...	Lowest

c. Find out the top 5 states with the highest & lowest average delivery time.

```
(select
c.customer_state,
round(avg(date_diff(o.order_delivered_customer_date
,o.order_purchase_timestamp, day)),2) as delivery_time
from `case_study.order_items` as oi
inner join `case_study.orders` as o
on oi.order_id = o.order_id

inner join `case_study.customers` as c
on c.customer_id = o.customer_id
group by c.customer_state
order by delivery_time desc
limit 5)

union all

(select
c.customer_state,
round(avg(date_diff(o.order_delivered_customer_date
,o.order_purchase_timestamp, day)),2) as delivery_time
from `case_study.order_items` as oi
inner join `case_study.orders` as o
on oi.order_id = o.order_id

inner join `case_study.customers` as c
on c.customer_id = o.customer_id
group by c.customer_state
order by delivery_time
limit 5)
```

Row	customer_state ▼	delivery_time ▼
1	RR	27.83
2	AP	27.75
3	AM	25.96
4	AL	23.99
5	PA	23.3
6	SP	8.26
7	PR	11.48
8	MG	11.52
9	DF	12.5
10	SC	14.52

- d. Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.

```

SELECT
    c.customer_state,
    AVG(DATE_DIFF(o.order_estimated_delivery_date,
o.order_delivered_customer_date, day)) AS avg_days_early
FROM `case_study.orders` o
JOIN `case_study.customers` c ON o.customer_id = c.customer_id
WHERE o.order_status = 'delivered'
    AND o.order_delivered_customer_date IS NOT NULL
GROUP BY c.customer_state
ORDER BY avg_days_early DESC
LIMIT 5;

```

Row	customer_state	avg_days_early
1	AC	19.76249999999...
2	RO	19.13168724279...
3	AP	18.73134328358...
4	AM	18.60689655172...
5	RR	16.41463414634...

vi. Analysis based on the payments:

A. Find the month on month no. of orders placed using different payment types.

```
select
p.payment_type,
format_date("%Y-%m", o.order_purchase_timestamp) as year_month,
count(distinct o.order_id) as order_count
from `case_study.orders` as o
inner join `case_study.payments` as p
on o.order_id = p.order_id
group by format_date("%Y-%m", o.order_purchase_timestamp),
payment_type
order by year_month
```


Row	payment_type ▾	year_month ▾	order_count ▾
1	credit_card	2016-09	3
2	UPI	2016-10	63
3	debit_card	2016-10	2
4	voucher	2016-10	11
5	credit_card	2016-10	253
6	credit_card	2016-12	1
7	credit_card	2017-01	582
8	voucher	2017-01	33
9	debit_card	2017-01	9
10	UPI	2017-01	197
11	UPI	2017-02	398
12	debit_card	2017-02	13
13	voucher	2017-02	69
14	credit_card	2017-02	1347

B. Find the no. of orders placed on the basis of the payment installments that have been paid.

```
select
p.payment_installments,
count(distinct o.order_id) as order_count
from `case_study.orders` as o
inner join `case_study.payments` as p
on o.order_id = p.order_id
where p.payment_installments >= 1
group by p.payment_installments
order by p.payment_installments desc
```

Row	payment_installm...	order_count ▼	
1	24	18	
2	23	1	
3	22	1	
4	21	3	
5	20	17	
6	18	27	
7	17	8	
8	16	5	
9	15	74	
10	14	15	
11	13	16	
12	12	133	

- 1.) Target has established a significant presence in Brazil, fulfilling orders across 29 states and 4119 unique cities.
- 2.) The state of SP serves as the primary hub with a massive customer base of 41,746, while **GO** represents a smaller market segment with 2,020 customers.
- 3.) The business reached a historic peak in November 2017, processing a record 7.54K orders in that single month.
- 4.) On a cumulative monthly basis, **August** stands out as the highest-performing month overall, with 10.84K total orders placed.
- 5.) When broken down by state and month, **SP** reached a peak of 4,982 orders in August, whereas **PR** saw its lowest activity in February with only 460 orders.
- 6.) The organization experienced explosive financial growth, with the total cost of orders surging by **136.97%** between 2017 and 2018 (measured from January to August).
- 7.) **SP** generated the highest total revenue with a combined order value of \$5,202,955.05\$, while **RR** contributed the minimum total value at \$7,829.43\$
- 8.) SP has the highest total volume, it has the lowest average order value at \$125.75\$; conversely, **PB** boasts the highest average order value at \$216.67\$, creating a price variance of \$90.91\$ between the two states.
- 9.) The most preferred payment method is the **Credit Card**, used for 76,505 transactions.
- 10.) **UPI** is the second most popular method, utilized 19,784 times